

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 5

Roll no:06 PRN No: 12413462 Name of Student: Purva Patil

Subject Name : Deep Learning

Problem Statement :

Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

Algorithm :

1. Import the required Python libraries.
2. Load a sequence classification dataset, such as the IMDB movie-review dataset, using TensorFlow/Keras.
3. Explore the dataset and examine the number of training and testing samples and their corresponding class labels.
4. Preprocess the text sequences by converting words into integer sequences and limiting the vocabulary size.
5. Pad or truncate the sequences to a fixed length so that all input sequences have the same size.
6. Split the dataset into training and testing sets.
7. Create an RNN model using an Embedding layer, SimpleRNN layer, and Dense output layer.
8. Train the RNN model using the training data.
9. Create an LSTM model using an Embedding layer, LSTM layer, and Dense output layer.
10. Train the LSTM model using the same training data and parameters.
11. Create a GRU model using an Embedding layer, GRU layer, and Dense output layer.
12. Train the GRU model using the same training data and parameters.
13. Predict the class labels for the test data using all three trained models.
14. Evaluate the models using appropriate metrics such as accuracy and loss.
15. Compare the performance of RNN, LSTM, and GRU based on their test accuracy and loss.
16. Visualize the performance of the three models using accuracy/loss comparison graphs.

Code :

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
```

```

from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU, Dense

vocab_size = 10000

(X_train, y_train), (X_test, y_test) = imdb.load_data(
    num_words=vocab_size
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("First review:", X_train[0])
print("First label:", y_train[0])

max_length = 200

X_train = pad_sequences(
    X_train,
    maxlen=max_length,
    padding='post',
    truncating='post'
)

X_test = pad_sequences(
    X_test,
    maxlen=max_length,
    padding='post',
    truncating='post'
)

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)

def create_rnn_model():
    model = Sequential([
        Embedding(vocab_size, 32),
        SimpleRNN(32),
        Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model

rnn_model = create_rnn_model()

```

```

rnn_history = rnn_model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2,
    verbose=1
)

rnn_loss, rnn_accuracy = rnn_model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("RNN Test Accuracy:", round(rnn_accuracy * 100, 2), "%")
print("RNN Test Loss:", round(rnn_loss, 4))

def create_lstm_model():
    model = Sequential([
        Embedding(vocab_size, 32),
        LSTM(32),
        Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model

lstm_model = create_lstm_model()

lstm_history = lstm_model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2,
    verbose=1
)

lstm_loss, lstm_accuracy = lstm_model.evaluate(
    X_test,
    y_test,
    verbose=0
)

```

```

)

print("LSTM Test Accuracy:", round(lstm_accuracy * 100, 2), "%")
print("LSTM Test Loss:", round(lstm_loss, 4))

def create_gru_model():
    model = Sequential([
        Embedding(vocab_size, 32),
        GRU(32),
        Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model

gru_model = create_gru_model()

gru_history = gru_model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.2,
    verbose=1
)

gru_loss, gru_accuracy = gru_model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("GRU Test Accuracy:", round(gru_accuracy * 100, 2), "%")
print("GRU Test Loss:", round(gru_loss, 4))

models = ['RNN', 'LSTM', 'GRU']

accuracies = [
    rnn_accuracy * 100,
    lstm_accuracy * 100,
    gru_accuracy * 100
]

plt.figure(figsize=(8, 5))

```

```
plt.bar(models, accuracies)

plt.xlabel("Model")
plt.ylabel("Test Accuracy (%)")
plt.title("RNN vs LSTM vs GRU")
plt.ylim(0, 100)

plt.show()

print("===== MODEL COMPARISON =====")

print(
    "RNN - Accuracy:",
    round(rnn_accuracy * 100, 2),
    "% | Loss:",
    round(rnn_loss, 4)
)

print(
    "LSTM - Accuracy:",
    round(lstm_accuracy * 100, 2),
    "% | Loss:",
    round(lstm_loss, 4)
)

print(
    "GRU - Accuracy:",
    round(gru_accuracy * 100, 2),
    "% | Loss:",
    round(gru_loss, 4)
)

plt.figure(figsize=(8, 5))

plt.plot(
    rnn_history.history['val_accuracy'],
    label='RNN'
)

plt.plot(
    lstm_history.history['val_accuracy'],
    label='LSTM'
)

plt.plot(
    gru_history.history['val_accuracy'],
    label='GRU'
)
```

```
plt.xlabel("Epochs")
plt.ylabel("Validation Accuracy")
plt.title("Validation Accuracy Comparison")
plt.legend()
plt.grid(True)

plt.show()
```

Output :

```
Epoch 1/5
157/157 ————— 18s 105ms/step - accuracy: 0.5120 - loss: 0.6926 - val_accuracy: 0.5120 - val_loss: 0.6915
Epoch 2/5
157/157 ————— 17s 111ms/step - accuracy: 0.5580 - loss: 0.6948 - val_accuracy: 0.5332 - val_loss: 0.6864
Epoch 3/5
157/157 ————— 17s 106ms/step - accuracy: 0.6026 - loss: 0.6545 - val_accuracy: 0.5904 - val_loss: 0.6750
Epoch 4/5
157/157 ————— 22s 116ms/step - accuracy: 0.6866 - loss: 0.5754 - val_accuracy: 0.7694 - val_loss: 0.5499
Epoch 5/5
157/157 ————— 19s 121ms/step - accuracy: 0.6738 - loss: 0.5836 - val_accuracy: 0.7886 - val_loss: 0.4871
```





